

SECURITY AUDIT

Symbiotic

Async/Account

FINAL



REPORT



Disclaimer:

Security assessment projects are time-boxed and reliant on information provided by the client, its affiliates, or its partners. The findings in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase, and apply only to the specific commit hash, version, or deployment reviewed at the time of the audit. Any subsequent modification, redeployment, or upgrade is outside the scope of this report and may invalidate its findings.

The audit does not constitute investment, legal, financial, regulatory, or tax advice, nor an endorsement of any project or team. It is based on a limited review of materials provided and is delivered on an "as-is," "where-is," and "as-available" basis. Use of blockchain technology is subject to unknown risks and flaws. The scope of this assessment is technical and contract-level; unless explicitly stated otherwise, it does not cover economic or game-theoretic risks, MEV, oracle manipulation, governance attacks, centralization or admin-key risks, off-chain components, front-end code, or the behavior of third-party protocols, dependencies, libraries, or services interacted with by the audited code.

We make no warranties, express or implied, regarding the accuracy, reliability, completeness, or availability of the report or any associated services, and disclaim all implied warranties including merchantability, fitness for a particular purpose, and non-infringement. We assume no responsibility for any product, service, software, code, or material referenced by, linked to, or accessible through the report.

This report is prepared solely for the contract owner and may not be relied upon by any third party. The contract owner is responsible for making their own decisions based on the report and should seek additional professional advice as needed. To the maximum extent permitted by law, our total aggregate liability arising from or related to this audit shall not exceed the fees actually paid for the engagement. The contract owner agrees to indemnify and hold harmless the audit firm and its personnel from any claims, damages, expenses, or liabilities arising from use of or reliance on the report or the audited smart contract.

This disclaimer and any dispute arising from the audit shall be governed by the laws of Wyoming (USA), without regard to its conflict-of-laws principles. By engaging in this audit, the contract owner acknowledges and agrees to these terms.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Symbiotic – Async/Account – Audit Report
Website	symbiotic.fi/
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/symbioticfi/core-mirror
Resolution 1	

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	2	1		1		
Low	5	1	1	3		
Informational	3	1		2		
Governance						
Total	13	3	1	6		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

3.1

Scope 1

Project	Symbiotic – Async/Account – Audit Report
Github repository	https://github.com/symbioticfi/core-mirror/blob/8fbde1bccc4400fc952f1484478209144dad8f06/src/contracts/adapters/Il-adapter/MidasAccount.sol
Resolution 1	https://github.com/symbioticfi/core-mirror/blob/32ade1b36ccc2472668382723c39e4eb97a3cced/src/contracts/adapters/Il-adapter/MidasAccount.sol

CutoffMidasAccount

The `CutoffMidasAccount` contract is a Midas redemption account variant that groups pending redemption requests into monthly cutoff cohorts instead of pricing them continuously. Each request is stamped with the bucket active at submission time, and once a cohort's cutoff date passes, all requests in that bucket freeze at the last Chainlink price printed before the cutoff rather than continuing to track the live oracle rate. It combines `MidasAccount`, which handles the underlying redemption-vault plumbing, with `CutoffAccount`, which supplies the generic bucket/timestamp mixin.

Invariants

- Every entry in `requestToBucket` corresponds to the `currentBucket()` value at the time that request was submitted.
- New redemption requests are only submitted outside the owner bypass when within `PRE_CUTOFF_WINDOW` of the next cutoff and past `COOLDOWN` since the last request.

Issue_01	<code>bucketToTimestamp</code> month-boundary corruption when <code>CUTOFF_DAY > 28</code>
Severity	Medium
Description	In <code>bucketToTimestamp</code>, the starting timestamp of a bucket is computed. However, if the <code>CUTOFF_DAY</code> is greater than 28, then erroneous timestamps are calculated for all months that do not have <code>CUTOFF_DAY</code> number of days. (February for 29, multiple months for 31). Say the initial timestamp is set to 31st January 2026, 00:00. Then, for bucket = 2, <code>bucketToTimestamp</code> calculates as follows: $month = 1 + 2 - 2 = 1$ $date_year = 2026$ $date_month = 2$ $date_day = 31$ $date_hour = 0$ But calling <code>DateTimeLib.dateTimeToTimestamp(2026,2,31,0,0)</code> returns 3rd March instead. Thus, there is a discrepancy and bucket boundaries overlap during the period 1-3rd March.
Recommendations	Ensure <code>CUTOFF_DAY < 28</code> in the constructor.
Comments / Resolution	Fixed by ensuring <code>CUTOFF_DAY <= 28</code> in the constructor.

Issue_02	mGLOBAL sync can revert on sub-minimum balances or Midas access-control failures, blocking finalization and swaps
Severity	Medium
Description	CutoffMidasAccount requests redemption whenever the sync path decides a request should be submitted and the account has a positive mGLOBAL balance. It does not check whether the balance satisfies Midas minimum redemption requirements before calling the Midas redemption vault. The Midas redemption request can revert when the balance is below the minimum amount, when the redemption function is paused, or when access-control checks fail. Because sync performs request finalization and request submission in the same transaction, a submission revert also rolls back any finalization that happened earlier in the same call. This also affects swaps. LiquidLaneAdapter transfers mGLOBAL into the account and then calls sync inline. If sync reverts because the resulting balance cannot be submitted to Midas, the whole swap reverts.
Recommendations	Check the Midas minimum redemption amount before attempting to submit a request. If the balance is below the minimum, skip request creation but still allow finalization of existing requests.
Comments / Resolution	Acknowledged.

Issue_03	Permissionless sync can grief real mGLOBAL redemptions
Severity	Low
Description	<code>CutoffMidasAccount.sync()</code> is permissionless. During the final cooldown period before a monthly cutoff, anyone can send a small redeemable amount of mGLOBAL to the account and call <code>sync()</code>. This creates a small redemption request and updates <code>lastRequestTimestamp</code>. If a real swap later sends a much larger mGLOBAL amount to the account before the cutoff, the adapter's <code>sync()</code> call cannot create a new redemption request because the cooldown is still active. The swap still pays the user, but the larger mGLOBAL balance remains idle in the account. As a result, that larger amount can miss the current monthly redemption cohort and be delayed until the next cycle unless the owner manually calls <code>sync()</code> before the cutoff.
Recommendations	Consider making the sync function admin-only, or documenting and acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_04	Round pricing discrepancy
Severity	Low
Description	The documentation states that “<i>pending requests compound until the cohort pricing date, then freeze at the first vault-feed print at/after it.</i>” This is inaccurate. In the current implementation, the cohort price is set at the last valid price before the end of the cohort, not the first price available after the end. This is evident from the loop shown below. <pre>uint48 nextBucketTimestamp = bucketToTimestamp(requestToBucket[requestId] + 1); while (timestamp >= nextBucketTimestamp) { - -roundId... // last oracle price of the cohort</pre>
Recommendations	Consider fixing the comment to reflect the price recording behaviour.
Comments / Resolution	Fixed by changing the comment.

Issue_05	Unbounded backward round walk can be gas-intensive
Severity	Low
Description	When valuing pending assets, the contract iterates over multiple chainlink rounds to find the accurate price, working backwards. <pre> while (timestamp >= nextBucketTimestamp) { [, answer,, timestamp,] = AggregatorV3Interface[aggregator].getRoundData[--roundId]; } </pre> Even within a single round, this loop can walk thousands of iterations for a feed that updates frequently, and if the cohorts are long. Thus, calling this function can become very gas-intensive, since there is no upper bound for the cost.
Recommendations	Consider having the admins cache the roundIds for pricing to fix the gas-cost of the function.
Comments / Resolution	Acknowledged.

Issue_06	Historical round used without staleness/validity checks
Severity	Low
Description	In <code>_pendingAssets</code>, the contract uses the historical price from the Chainlink aggregator directly to price the tokens undergoing redemption. The issue is that the raw price is used without any checks. The price could have been unupdated for a long time, and thus could have been too stale to use as the redemption price for the cohort. Chainlink documents that a zero <code>updatedAt</code> means the round is incomplete and must not be used. The project's own <code>ChainlinkPriceFeed</code> library already treats <code>updatedAt == 0</code> as invalid data. This path calls the aggregator directly and skips that check. The <code>Midas</code> vault oracle implements a bound check in its <code>getDataInBase18</code> function, which is also ignored here.
Recommendations	Consider adding a bound check for the historical oracle price, and ensure that the price being used has updates before and after it within a staleness threshold. Also, reject rounds with <code>updatedAt=0</code>, indicating incomplete rounds.
Comments / Resolution	Partially fixed by adding an <code>updatedAt</code> check.

Issue_07	CutoffMidasAccount skips Chainlink rounds published exactly at bucket cutoff
Severity	Low
Description	<code>CutoffMidasAccount.pendingAssets</code> prices pending requests by walking backward from <code>latestRoundData</code> until it finds a round with <code>updatedAt</code> strictly before <code>bucketToTimestamp(requestToBucket + 1)</code>. Monthly cutoff buckets are fixed on a calendar schedule, but Chainlink rounds are event-driven and may land at any second. Because the loop uses a strict less-than boundary, a round whose <code>updatedAt</code> equals the cutoff timestamp is skipped and the prior round's price is used. If Midas publishes NAV at or near the cutoff hour, the cohort locks an older rate instead of that cutoff print. The locked price therefore depends on second-level timing between oracle publication and bucket boundaries.
Recommendations	If exact-cutoff prints should count, adjust the boundary condition so rounds at the cutoff timestamp are included. Otherwise, document that cohort pricing uses the last Chainlink round strictly before the bucket cutoff, not the first at or after it, and confirm this matches Midas's intended redemption accounting.
Comments / Resolution	Acknowledged.

Issue_08	<code>totalAssets</code> can revert even when there is no <code>mGLOBAL</code> exposure
Severity	Informational
Description	<code>CutoffMidasAccount.totalAssets()</code> always reaches <code>_pendingAssets()</code>. Inside <code>_pendingAssets()</code>, the contract validates the current Midas oracle price before checking whether there are any pending requests. This means <code>totalAssets()</code> can revert if the current <code>mGLOBAL</code> oracle is stale, unavailable, or outside bounds, even when the account has no <code>mGLOBAL</code> balance and no pending redemption requests. This can unnecessarily break accounting reads for an account that has no exposure requiring an oracle price.
Recommendations	Only validate the oracle when there is a token balance or pending request that actually needs pricing.
Comments / Resolution	Fixed.

Issue_09	A single invalid datapoint can brick the entire contract
Severity	Informational
Description	In <code>_pendingAssets</code>, the loop iterates over multiple rounds and checks if it has a valid price. <pre> while (timestamp >= nextBucketTimestamp) { [, answer,, timestamp,] = AggregatorV3Interface[aggregator].getRoundData[--roundId]; } if (answer <= 0) { revert InvalidCutoffPrice(); } </pre> The issue is that if a single round has price = 0, it can immediately break the contract, since the execution will revert with <code>InvalidCutoffPrice</code>. This can happen if the walk eventually reads invalid or empty rounds.
Recommendations	Consider ignoring an invalid price and iterating more until a valid price is found, within a bound.
Comments / Resolution	Acknowledged.

Issue_10	Direct token transfers affect accounting
Severity	Informational
Description	<code>CutoffMidasAccount</code> inherits <code>Account.totalAssets()</code>, which counts the account's raw token balances directly. Because of this, if <code>USDC</code> or <code>mGLOBAL</code> is sent directly to the account, those tokens are immediately included in <code>totalAssets()</code>, even though they did not enter through the intended flow. This can affect the reported account value and may make accounting harder to reason about.
Recommendations	Document that direct token transfers to the account affect <code>totalAssets()</code> .
Comments / Resolution	Acknowledged.